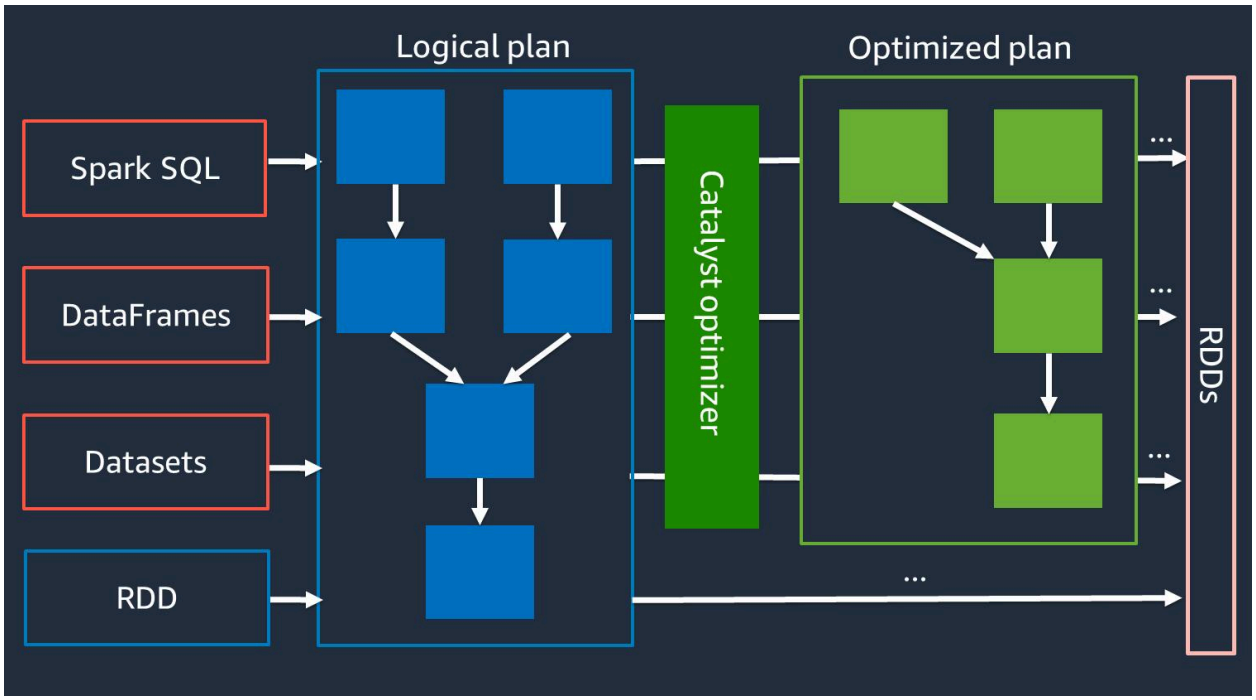
Spark SQL – Complete Training Material

This module covers Spark SQL and DataFrame operations, progressing from fundamentals to practical data engineering use cases. It is structured for your November 2026 Data Engineering training batch, with concepts, PySpark examples, SQL queries, and hands-on labs.



[Key topics in Apache Spark - AWS Prescriptive Guidance](docs.aws.amazon.com)
docs.aws.amazon.com

Intermediate
PySpark
Spark SQL

Suggested module duration: 1–2 training days

1. Introduction to Spark SQL

What is Spark SQL?

Spark SQL is a Spark module used to process structured and semi-structured data using SQL queries and DataFrame APIs.

It allows developers to work with structured data using:

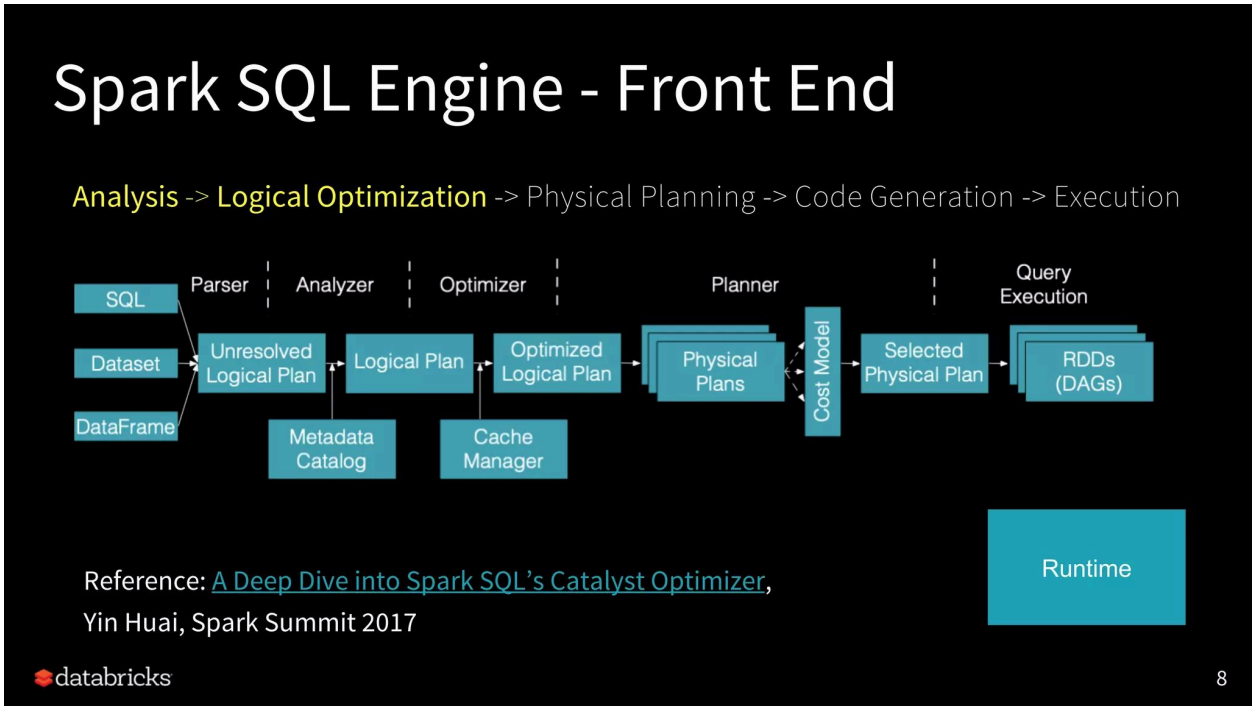
- SQL queries.
- DataFrame API.
- Dataset API (Scala and Java).
- External data sources.
- Hive integration.

Why Spark SQL?

Traditional SQL	Spark SQL
Primarily database-oriented	Distributed data processing
Often limited by database resources	Uses cluster resources
Works with relational data	Structured and semi-structured data
Database execution engine	Catalyst optimizer and Spark execution engine

Spark SQL architecture

A Deep Dive into Query Execution Engine of Spark SQL | PDF
slideshare.net



Important components:

1. SQL Parser – Parses SQL queries.
2. Analyzer – Resolves tables, columns and data types.
3. Catalyst Optimizer – Optimizes logical execution plans.
4. Physical Planner – Generates physical execution strategies.
5. Tungsten – Improves execution efficiency and memory management.
6. Spark Execution Engine – Executes tasks across the cluster.

Example:

```
None
df = spark.read.csv(    "employees.csv",    header=True,
inferSchema=True)df.createOrReplaceTempView("employees")spark.sql("
"""    SELECT department, COUNT(*) AS employee_count    FROM
employees    GROUP BY department""").show()
```

2. DataFrames and Spark SQL

A DataFrame is a distributed collection of data organized into named columns.

Example dataset:

emp_id	name	department	salary
101	Arun	IT	60000
102	Priya	HR	45000
103	Kumar	IT	75000
104	Divya	Finance	55000

Creating a DataFrame:

```
None
data = [    (101, "Arun", "IT", 60000),    (102, "Priya", "HR", 45000),    (103, "Kumar", "IT", 75000),    (104, "Divya", "Finance", 55000)]columns = ["emp_id", "name", "department", "salary"]df = spark.createDataFrame(data, columns)df.show()df.printSchema()
```

3. Creating DataFrames from different sources

Spark supports multiple data formats.

[CSV File Icon. Vector File Format. CSV File Extension Modern Flat Design]

CSV files

```
None
df = spark.read.option(    "header", True).option( "inferSchema", True).csv("employees.csv")
```

JSON files

```
None
df = spark.read.json("employees.json")
```

Apache Parquet

Parquet files

```
None
df = spark.read.parquet("employees.parquet")
```

ORC file format

ORC files

```
None
df = spark.read.orc("employees.orc")
```

Other supported sources include Avro, JDBC databases, Delta Lake and Hive tables (with appropriate support/configuration).

4. Schema Inference and Manual Schema Definition

A. Schema inference

Spark automatically determines column types.

```
None
df = spark.read \    .option("header", True) \
.option("inferSchema", True) \
.csv("employees.csv")df.printSchema()
```

Possible output:

```
None
root
 |-- emp_id: integer
 |-- name: string
 |-- department: string
```

```
 |-- salary: integer
```

B. Manual schema definition

Manual schemas give you control over data types and nullability.

```
None
from pyspark.sql.types import *schema = StructType([
StructField("emp_id", IntegerType(), False),    StructField("name",
StringType(), True),    StructField("department", StringType(),
True),    StructField("salary", DoubleType(), True)])df =
spark.read \    .schema(schema) \    .option("header", True) \
.csv("employees.csv")
```

Training discussion: Why should production ETL pipelines prefer explicit schemas over schema inference?

Answer: Explicit schemas provide predictable types, avoid inference overhead, improve consistency and help detect malformed data.

5. DataFrame Operations

A. select()

Select specific columns.

```
None
df.select("name", "salary").show()
```

B. filter() and where()

Filter records.

```
None
df.filter(df.salary > 50000).show()df.where("department =
'IT'").show()
```

C. withColumn()

Create or modify a column.

```
None
from pyspark.sql.functions import coldf = df.withColumn(
"annual_salary",    col("salary") * 12)
```

D. drop()

Remove columns.

```
None
df.drop("annual_salary").show()
```

E. groupBy() and aggregation

```
None
from pyspark.sql.functions import (    count, sum, avg, max,
min)df.groupBy("department").agg(
count("*").alias("employee_count"),
avg("salary").alias("average_salary"),
max("salary").alias("maximum_salary"),
min("salary").alias("minimum_salary")).show()
```

F. Sorting

```
None
df.orderBy(col("salary").desc()).show()
```

G. Joins

Suppose we have two DataFrames:

Employees

emp_id	name	dept_id

101	Arun	10
102	Priya	20
103	Kumar	10

Departments

dept_id	department
10	IT
20	HR
30	Finance

Join example:

```
None
result = employees.join(    departments,    employees.dept_id == departments.dept_id,    "inner")result.show()
```

Types of joins:

Join	Description
Inner	Matching records

Left	All left records and matching right records
Right	All right records and matching left records
Full Outer	All records from both sides
Left Semi	Left records having a match
Left Anti	Left records without a match
Cross	Cartesian product

H. Union and unionByName

```
None
df1.union(df2).show()
```

```
None
df1.unionByName(df2).show()
```

Remember:

- `union()` combines by column position.
- `unionByName()` combines by column name.
- Both retain duplicates.
- Use `distinct()` when duplicate removal is required.

6. Handling Missing Data

Missing values are common in real-world datasets.

```
None
# Identify null valuesdf.filter(col("salary").isNull()).show()# Remove records with null valuesdf.na.drop().show()# Replace null valuesdf.na.fill(0, ["salary"]).show()# Replace missing departmentdf.na.fill("Unknown", ["department"]).show()
```

Other useful methods:

```
None
df.dropna()df.fillna(0)df.replace("NA", None)
```

7. Working with Complex Data Types

Spark supports nested and complex data structures.

A. ArrayType

Example:

```
None
data = [    (1, "Arun", ["Python", "Spark", "SQL"]),    (2, "Priya", ["Java", "SQL"])]df = spark.createDataFrame(    data, ["id", "name", "skills"])df.show(truncate=False)
```

Access array elements:

```
None
from pyspark.sql.functions import coldf.select(    "name", col("skills")[0].alias("first_skill")).show()
```

Explode arrays:

```
None
from pyspark.sql.functions import explodedf.select(    "name", explode("skills").alias("skill")).show()
```

B. StructType

A struct represents nested records.

```
None
from pyspark.sql.functions import coldata = [    (1, "Arun",
("Chennai", "Tamil Nadu")),    (2, "Priya", ("Coimbatore", "Tamil
Nadu"))]df = spark.createDataFrame(    data,    ["id", "name",
"address"])df.select(    "name",
col("address._1").alias("city"),
col("address._2").alias("state")).show()
```

For production datasets, define named nested fields using `StructType` rather than relying on tuple-generated field names.

8. Working with Dates and Timestamps

Commonly used functions:

Function	Purpose
current_date()	Current date
current_timestamp()	Current timestamp
to_date()	Convert to date
to_timestamp()	Convert to timestamp
datediff()	Difference between dates

date_add()	Add days
date_sub()	Subtract days
year()	Extract year
month()	Extract month
dayofmonth()	Extract day

Example:

```
None
from pyspark.sql.functions import *df = spark.createDataFrame( [("Arun", "2026-11-05")],    ["name", "joining_date"])df = df.withColumn(    "joining_date",    to_date("joining_date", "yyyy-MM-dd"))df.select(    "name",    year("joining_date").alias("year"),    month("joining_date").alias("month"),    date_add("joining_date", 30).alias("after_30_days")).show()
```

For timestamp processing, also teach `spark.sql.session.timeZone`, especially when working with data from multiple countries.

9. SQL Queries Using spark.sql()

Spark allows you to execute SQL directly against registered views.

```
None
df.createOrReplaceTempView("employees")
```

Execute SQL:

```
None
result = spark.sql("""    SELECT *    FROM employees    WHERE
salary > 50000""")result.show()
```

Aggregate:

```
None
spark.sql("""    SELECT department,           COUNT(*) AS
total_employees,           AVG(salary) AS average_salary    FROM
employees    GROUP BY department""").show()
```

Window functions:

```
None
spark.sql("""    SELECT name,           department,
salary,           RANK() OVER (               PARTITION BY
department               ORDER BY salary DESC           ) AS
salary_rank    FROM employees""").show()
```

10. Temporary and Global Views

Feature	Temporary View	Global Temporary View
Scope	Current SparkSession	Shared across SparkSessions in the same application
Lifetime	SparkSession	Spark application
Access	View name	global_temp.view_name

Persistence	No	No

Temporary view:

```
None
df.createOrReplaceTempView("employees")spark.sql("""    SELECT * FROM employees""").show()
```

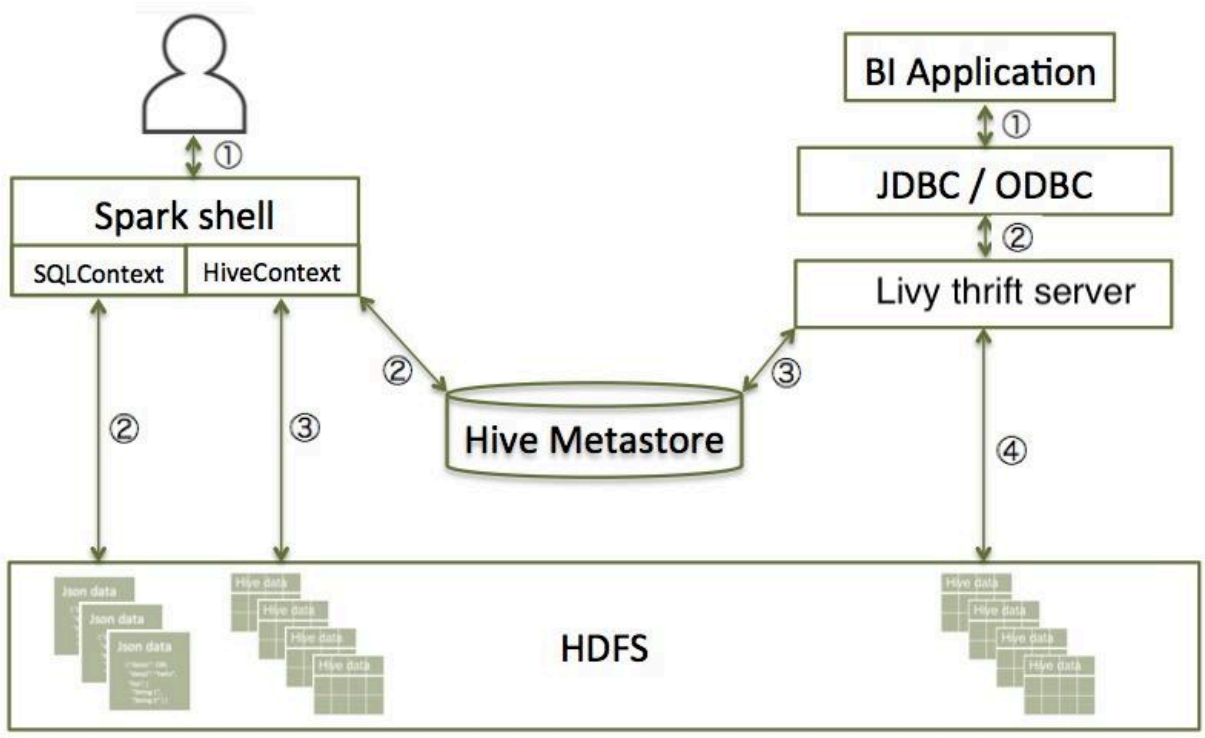
Global temporary view:

```
None
df.createOrReplaceGlobalTempView("employees")spark.sql("""
SELECT * FROM global_temp.employees""").show()
```

Important: Neither temporary nor global temporary views persist after their respective lifetimes.

11. Integrating Spark SQL with Hive

Using Spark SQL
docs.cloudera.com



Spark SQL can interact with Hive tables using Hive Metastore integration.

Step 1: Enable Hive support

```
None
from pyspark.sql import SparkSessionspark = SparkSession.builder \
.appName("SparkHiveIntegration") \    .enableHiveSupport() \
.getOrCreate()
```

Step 2: List databases

```
None
spark.sql("SHOW DATABASES").show()
```

Step 3: Create a database

```
None
spark.sql("""    CREATE DATABASE IF NOT EXISTS training_db""")
```

Step 4: Use the database

```
None
spark.sql("USE training_db")
```

Step 5: Create a Hive table

```
None
spark.sql("""    CREATE TABLE IF NOT EXISTS employees (
emp_id INT,        name STRING,        department STRING,
salary DOUBLE    )    STORED AS PARQUET""")
```

Step 6: Insert records

```
None
spark.sql("""    INSERT INTO employees    VALUES    (101, 'Arun',
'IT', 60000),    (102, 'Priya', 'HR', 45000)""")
```

Step 7: Query Hive table

```
None
spark.sql("""    SELECT department,           AVG(salary) AS
average_salary    FROM employees    GROUP BY department""").show()
```

Important configuration topics:

- `hive-site.xml`
- Hive Metastore URI
- `spark.sql.catalogImplementation`
- Hadoop configuration
- Warehouse directory
- Hive-compatible table formats

Note: `enableHiveSupport()` enables Hive integration, but does not automatically start Hive Metastore or HiveServer2. Spark needs the correct metastore configuration and compatible dependencies.

12. Hands-on Lab Exercises

Practical training

Use one common Employee dataset for the exercises.

Lab 1 – DataFrame creation

1. Create a DataFrame with 10 employees.
2. Define a manual schema.
3. Print the schema.
4. Display the first five records.
5. Count total employees.

Lab 2 – DataFrame transformations

1. Select employee name and salary.
2. Filter employees earning more than 50,000.
3. Add annual salary.
4. Find the highest-paid employee.
5. Sort employees by salary.

Lab 3 – Aggregations and joins

1. Calculate department-wise employee count.
2. Find average salary by department.
3. Join employee and department datasets.
4. Find employees without matching departments.
5. Perform a full outer join.

Lab 4 – Complex data

1. Create an array column containing employee skills.
2. Extract the first skill.
3. Explode the skills array.
4. Create a nested address struct.
5. Extract city and state from the struct.

Lab 5 – Spark SQL

1. Create a temporary view.
2. Find the top three highest-paid employees.
3. Calculate department-wise average salary.
4. Rank employees by salary within departments.
5. Create and query a global temporary view.

Lab 6 – Hive integration

1. Enable Hive support.
2. Connect to Hive Metastore.
3. List databases.
4. Create a database.
5. Create a managed table.
6. Insert records.
7. Query the table using Spark SQL.
8. Verify the data using Beeline.

13. Interview Questions

No.	Question
1	What is Spark SQL?
2	What is the difference between RDD and DataFrame?
3	What is Catalyst Optimizer?
4	What is Tungsten?
5	What is schema inference?
6	Why use an explicit schema?
7	Difference between select and withColumn?
8	Difference between filter and where?

9 Difference between union and unionByName?

10 What is a broadcast join?

11 What is a temporary view?

12 What is a global temporary view?

13 How does Spark SQL integrate with Hive?

14 What is the difference between Parquet and ORC?

15 How does Spark handle null values?

16 What are ArrayType and StructType?

17 What is predicate pushdown?

18 What is partition pruning?

19 What is the difference between managed and external tables?

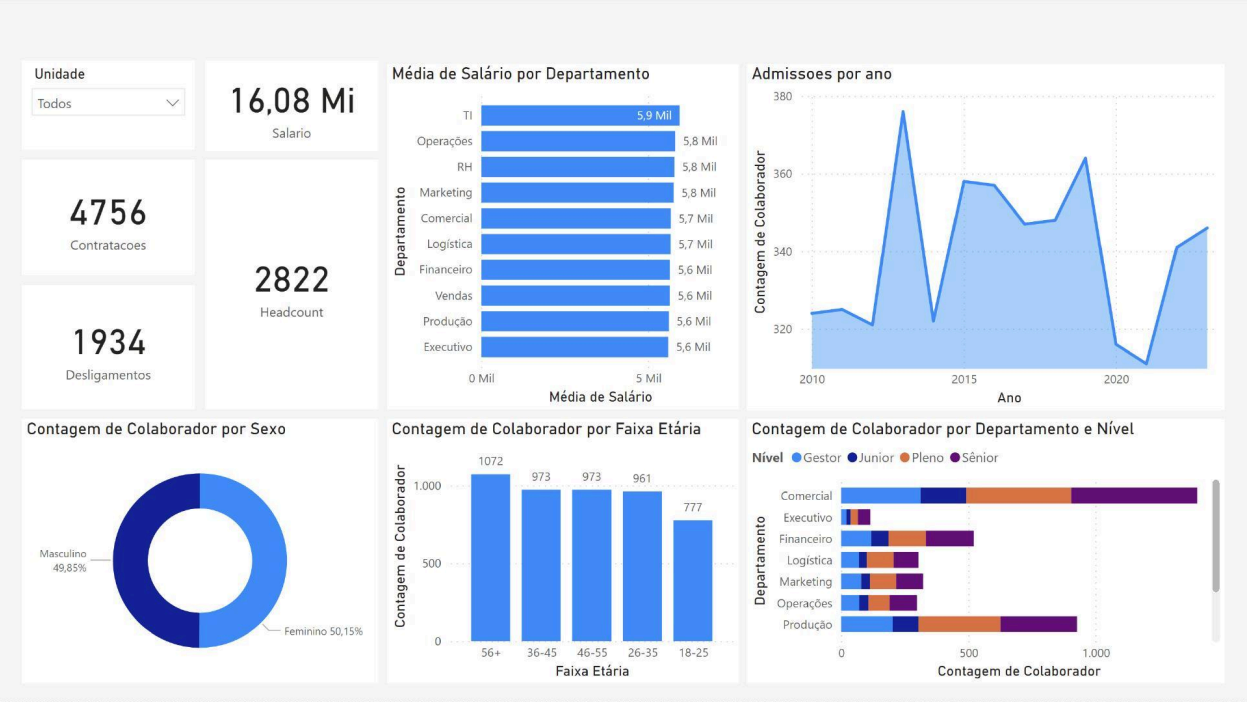
20 How do you optimize a Spark SQL query?

14. Recommended Training Sequence

Session	Topics	Practical
Day 1 – Session 1	Spark SQL architecture, DataFrames	DataFrame creation
Day 1 – Session 2	Schema, CSV, JSON, Parquet	File ingestion
Day 1 – Session 3	Transformations and aggregations	Employee analysis
Day 2 – Session 1	Joins, unions, null handling	Data integration
Day 2 – Session 2	Complex types, dates, timestamps	Nested JSON
Day 2 – Session 3	SQL queries, views, window functions	SQL analytics
Day 2 – Session 4	Hive integration	Hive hands-on

15. Capstone Project – Employee Analytics using Spark SQL

Grátis: Plano de fundo editável - Dashboard RH - Material Claro e Objetivo em PDF para Estudo Rápido
passeidireto.com



Business scenario: An organization receives employee data from multiple sources and wants to build an employee analytics platform.

Input:

- `employees.csv`
- `departments.json`
- `employee_skills.json`
- `attendance.parquet`

Expected output:

1. Employee master DataFrame.
2. Department-wise salary analysis.
3. Employee attendance analysis.
4. Missing data quality report.
5. Skills distribution report.
6. Top-performing employees.
7. Store processed data in Parquet.
8. Create Hive tables for analytical reporting.

Expected architecture:

```
None
CSV / JSON / Parquet
          |
          v
   Spark DataFrames
          |
          v
   Data Cleansing
          |
          v
  Spark Transformations
          |
          v
     Spark SQL
          |
          v
   Hive Metastore
          |
          v
    Parquet Tables
          |
          v
   BI / Analytics
```

Trainer's recommendation

For this module, I recommend emphasizing three things:

1. DataFrame API vs SQL: Make trainees solve the same problem using both approaches.
2. Execution understanding: Teach `explain("formatted")`, logical plans, physical plans, and Catalyst optimization.
3. Real-world ETL: Use nested JSON, Parquet, joins, aggregations, and Hive tables rather than limiting the lab to employee examples.

This will help trainees understand not just how to write Spark SQL, but how Spark processes data in production.